浙江大學



Course Project Report on Social Privacy Networks

Black-Box Reverse Modeling and Prediction of Google Play Age Rating Questionnaire

Group Members:	Bofan Chen, Yuhan Chen, Zihan Chen, Jiashun Shi, Zixuan Wang
Project Repository:	https://github.com/CYH0415/OSN-Lab-Main
Experimental Subject:	Google Play / IARC Content Rating Questionnaire
Prediction Target:	North American ESRB Age Rating `rating__north_america`
Core Methodology:	Automated Sampling, Data Cleaning, Feature Engineering, Multi-Class Machine Learning, Model Interpretability Analysis

July 1, 2026

Contents

1 Abstract **3**

2 Course Requirements and Project Completion **3**
2.1 Understanding Project Tasks . . . 3
2.2 Project Completion Status . . . 5

3 Experimental Background and Problem Definition **5**
3.1 Google Play Age Rating Mechanism . . . 5
3.2 Black-Box Reverse Modeling Goal . . . 6

4 System Architecture and Repository Structure **6**
4.1 Code Repository Structure . . . 6
4.2 Technology Stack . . . 7

5 Questionnaire Structure Inspection and Sampling Design **8**
5.1 Questionnaire Structure Inspection . . . 8
5.1.1 Google Questionnaire Structural Features . . . 8
5.1.2 DFS Structure Inspection Algorithm . . . 9
5.2 Sampling Strategy . . . 10

6 Automated Data Collection **11**
6.1 Automated Workflow . . . 11
6.2 Core Automation Code . . . 11
6.3 Data Quality and Collection Failures . . . 11

7 Data Cleaning and Dataset Analysis **12**
7.1 Cleaning Principles . . . 12
7.2 Train/Test Split . . . 12
7.3 Data Distribution Visualization . . . 13

8 Feature Engineering **14**
8.1 Input and Output Definition . . . 14
8.2 Encoding Scheme . . . 14
8.3 Core Feature Engineering Code . . . 15

9 Model Training and Parameter Tuning **15**
9.1 Model Selection . . . 15
9.2 Logistic Regression and Random Forest . . . 16
9.3 LightGBM and XGBoost . . . 17

10 Experimental Results and Model Comparison **17**
10.1 Overall Performance . . . 17
10.2 Generalization Gap . . . 18
10.3 Classification Report of the Best Model . . . 18
10.4 Confusion Matrix Analysis . . . 18

11 In-depth Analysis **20**
11.1 Why L1 Logistic Regression Performs Best . . . 20
11.2 Core Influential Factors . . . 20
11.3 Significance of Structural Missingness . . . 21
11.4 Questionnaire Redundancy and Minimal Feature Sets . . . 21
11.5 Gray Areas in Rating Boundaries . . . 22

12 Experimental Challenges and Solutions **22**
12.1 Data Collection Challenges 22
12.2 Modeling Challenges 23

13 Group Division of Labor **23**

14 Major Project Findings **24**

15 Limitations and Future Directions **24**
15.1 Current Limitations 24
15.2 Future Directions 24

16 Conclusion **25**

A Experiment Replication Commands **25**

B Key Output Files **25**

C References **26**

1 Abstract

The age rating mechanism on Google Play helps parents and general users determine if an application is appropriate for minors. When developers fill out the IARC content rating questionnaire in the Google Play Developer Console, the system generates age ratings, content descriptors, and interactive elements for various territories. However, Google does not publish the exact decision rules mapping questionnaire answers to final ratings. Developers and users can only observe the inputs (questionnaire answers) and outputs (rating results) without direct access to how each question, branch, and risk factor influences the final rating. Therefore, this project treats the Google Play age rating questionnaire as a black-box system and employs automated sampling and machine learning to reverse model the mapping from questionnaire answers to the final age ratings.

In line with the course requirements, this project has successfully implemented tree-structured questionnaire inspection, automated sample submission, data cleaning and validation, feature engineering, multi-model training, model performance comparison, and rating mechanism analysis. The final dataset contains 1,265 valid samples covering three primary application categories: "All Other App Types", "Game", and "Social or Communication". The cleaned data is deterministically split into 1,012 training samples and 253 testing samples. The feature engineering stage encodes 165 raw input fields into 758-dimensional features. Specifically, 133 radio/categorical fields are one-hot encoded, and 32 checkbox fields are multi-hot encoded. Structural missingness indicators such as `__INACTIVE__`, `__NOT_APPLICABLE__`, and `__NONE__` are explicitly retained.

In the model training phase, we trained and compared four types of models: L1-regularized Logistic Regression, Random Forest, LightGBM, and XGBoost. Experimental results show that L1-regularized Logistic Regression achieves the best performance on the test set, with an Accuracy of 0.921, a Macro F1-score of 0.918, and a Macro One-vs-Rest AUC of 0.977. XGBoost ranks second with 0.814 Accuracy and approximately 0.810 Macro F1. After parameter tuning, Random Forest reaches 0.783 Accuracy and 0.797 Macro F1. LightGBM exhibits relatively weaker performance under this sample size and high-dimensional sparse feature setting, yielding an Accuracy of 0.680. Further analysis reveals that Google Play age ratings are primarily driven by high-risk content such as violence/gore severity, sexual content/nudity, drug references, and gambling/cash rewards. Furthermore, the tree structure of the questionnaire itself carries critical information, as the activation of certain sub-questions implicitly reflects the underlying risk path.

This report documents the experimental background, project completion status, system architecture, data collection strategy, data quality control, feature engineering, model training and tuning, results comparison, encountered challenges and solutions, group member division of labor, and directions for future improvement.

2 Course Requirements and Project Completion

2.1 Understanding Project Tasks

According to the course project guidelines, this project reverse models the Google Play age rating questionnaire as a black box. The core requirements consist of four parts:

1. **Data Collection Strategy & Execution**: Design a systematic and efficient sampling strategy for the tree-structured content rating questionnaire, automatically submit questionnaires and collect rating results using scripts, and acquire at least 1,000 valid and diverse samples.

2. **Model Training & Prediction**: Perform feature engineering on the collected questionnaire data, train multi-class age rating prediction models, compare at least three different model types, and optimize their performance.

3. **Results Analysis**: Analyze overall model performance, investigate prediction differences among age rating classes, and explore the hidden rules within the Google Play rating mechanism (e.g., finding the most influential questions or content risk factors).

4. **Comprehensive Report**: Compile a complete report covering the data collection strategy, dataset analysis, model training, performance evaluation, challenges and solutions, key findings, core code snippets, and crucial charts.

The implementation goes beyond simple modeling on a static dataset, establishing a reproducible data closed-loop starting from automated web page inspection. The overall workflow is illustrated in Figure 1.

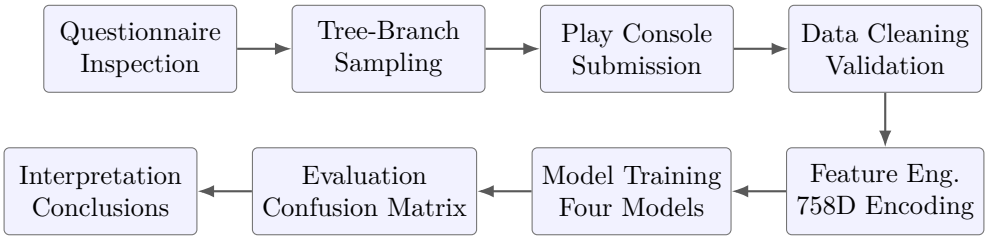


Figure 1: End-to-end experiment workflow. The project forms a closed loop starting from dynamic questionnaire inspection, through sampling, collection, cleaning, encoding, modeling, and interpretability analysis.

2.2 Project Completion Status

Table 1: Comparison of Course Requirements and Project Completion

Requirement	Project Completion Status	Status
Tree Questionnaire Understanding	Used Playwright and Chrome CDP to inspect questions, options, parent-child dependencies, and branches for the three main categories. Generated structured outputs such as `question_graph.json`, `question_graph.md`, and `question_graph.html`.	Completed
Automated Data Collection	Wrote scripts including `inspect_questionnaire.mjs`, `run_rating_samples.mjs`, and `autonomous_rating_sampler.mjs` to reuse a real Chrome session for automatic questionnaire filling, submission, parsing, and result saving.	Completed
$\geq$ 1,000 Valid Samples	The cleaned dataset contains 1,265 valid samples (satisfying the 1,000 requirement). We also recorded 770 execution failures, 30 upstream quarantined samples, and 13 rating conflicts for audit purposes.	Completed
Feature Engineering	Excluded all rating labels except the target column to prevent data leakage. Kept only `category` and `answer__*` to construct a 758-dimensional feature space from 165 raw inputs.	Completed
$\geq$ 3 Prediction Models	Trained and compared four models: Logistic Regression, Random Forest, LightGBM, and XGBoost.	Completed
Tuning	Optimized hyperparameters and applied regularizations for all models. Compared Accuracy, Macro F1, Weighted F1, and Macro AUC.	Completed
Results Analysis	Analyzed overall accuracy, category-level differences, confusion matrices, feature importance, structural missingness, and model redundancy.	Completed
Comprehensive Report	Compiled this complete report using LaTeX, incorporating core code, statistics, and figures.	Completed

3 Experimental Background and Problem Definition

3.1 Google Play Age Rating Mechanism

Applications published on Google Play are required to complete a content rating questionnaire provided by the IARC system. Developers must answer whether the application contains violence, gore, horror, sexual content, drugs, gambling, user interaction, location sharing, or digital purchases. Based on these answers, the system generates ratings for different jurisdictions (e.g., North American ESRB, European PEGI, German USK, Australian ACB, etc.).

This project selects the North American ESRB rating as the primary prediction target, namely `rating__north_america`. This label contains five categories, as listed in Table 2.

Table 2: North American ESRB Age Rating Labels

Index	Label	Meaning
0	Everyone	Suitable for all ages
1	Everyone 10+	Suitable for ages 10 and older
2	Teen	Suitable for teens
3	Mature 17+	Suitable for ages 17 and older
4	Adults only 18+	Explicitly for adults aged 18 and older

3.2 Black-Box Reverse Modeling Goal

From a machine learning perspective, the Google Play age rating questionnaire can be abstracted as an unknown function:

$$f : X \to Y,$$

where X represents the questionnaire answer state and Y represents the final age rating label. Since the rules of f are hidden by Google, we query the system to collect input-output samples:

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}.$$

The goal of the project is to learn an approximation function $\hat{f}$ that can predict the ESRB rating given new questionnaire answers:

$$\hat{y} = \hat{f}(x).$$

Compared to standard tabular classification tasks, this task presents several unique challenges:

1. **Dynamic Tree Structure**: Different application categories trigger different sets of questions. Within a category, selecting a specific option determines whether subsequent sub-questions appear.

2. **Non-Random Missingness**: Untriggered sub-questions are not missing due to data corruption, but represent structural states dictated by questionnaire logic.

3. **Massive Sample Space**: Combinatorial explosion prevents exhaustive search of all radio and checkbox options, making manual sampling infeasible.

4. **Imbalanced Classes**: Intermediate categories (e.g., Everyone 10+) have fewer samples, and boundaries (e.g., Teen vs. Mature 17+) are highly ambiguous.

5. **Web Automation Hurdles**: Google Play Console page interactions involve login sessions, draft saving, loading delays, overlays, and asynchronous updates.

4 System Architecture and Repository Structure

4.1 Code Repository Structure

The code repository is organized into modular directories corresponding to web automation, data collection, cleaning, feature engineering, and model training. The main modules are summarized in Table 3.

Table 3: Key Modules in the Project Repository

Path	Purpose
`scripts/`	

Path	Purpose
`inspect_questionnaire.mjs`	Uses Playwright and Chrome CDP to inspect the dynamic question structures, options, and parent-child dependencies of the Google Play IARC questionnaire.
`scripts/run_rating_samples.mjs`	Connects to debug Chrome, fills out the questionnaire using specified samples, saves the draft, parses final ratings, and records results/failures.
`scripts/autonomous_rating_sampler.mjs`	Fully automated loop for candidate generation, batch selection, filling, parsing, and retrying to achieve the target 1,000 valid samples.
`data_categories/`	Contains inspected question graphs, questionnaire trees, interactive HTML visualizers, and risk annotations for the three categories.
`rating_artifacts/`	Stores raw collected samples, execution logs, failure events, evidence screenshots, and batch summaries.
`dataset_v2/`	Retains semantically deduplicated datasets, debug evidence, execution errors, quarantined samples, rating conflicts, and schemas.
`task3_data_cleaning/`	Python cleaning scripts, outputting `clean_dataset.csv`, `train.csv`, `test.csv`, the data dictionary, and cleaning reports.
`task4_feature_engineering/`	Scripts and features for encoding categorical/checkbox inputs into sparse matrices, outputting feature dictionaries, label mappings, and class weights.
`task5_1_lightgbm\&xgboost/`	Notebooks for training, tuning, evaluating, and plotting confusion matrices for LightGBM and XGBoost models.
`task5_2_logistic_regression\&random_forest/`	Scripts, models, classification reports, metrics, and confusion matrices for Logistic Regression and Random Forest.

4.2 Technology Stack

The project is implemented using Node.js and Python. The web automation components rely on Playwright and the Chrome DevTools Protocol. Data cleaning and model training are powered by pandas, scikit-learn, scipy sparse, joblib, LightGBM, and XGBoost.

Table 4: Project Technology Stack

Module	Technologies and Tools
Web Automation	Playwright, Chrome CDP, Dedicated Chrome Profile, Google Play Developer Console
Data Formats	JSONL, CSV, Schema JSON, Run Configs, Debug Evidence
Data Processing	Python, pandas, scipy sparse, deterministic hashing
Modeling	scikit-learn Logistic Regression, Random Forest, LightGBM, XGBoost
Evaluation	Accuracy, Macro F1, Weighted F1, Macro OvR AUC, classification reports, confusion matrices
Reporting	LaTeX, TikZ, booktabs, longtable

5 Questionnaire Structure Inspection and Sampling Design

5.1 Questionnaire Structure Inspection

The Google Play content rating questionnaire is not a static form, but a dynamic, tree-structured questionnaire. The project must first identify the questions, options, parent-child dependencies, and reachable paths for each application category. The usage of the inspection script is detailed in `docs/questionnaire-inspection.md`. Since Google blocks default Playwright headless login, we use a real Chrome instance connected via CDP:

Listing 1: Typical command for questionnaire structure inspection

```
# Start real Chrome with remote debugging and log into developer account
bash scripts/open_debug_chrome.sh

# Reuse the Chrome session to inspect the questionnaire structure
CDP_URL="http://127.0.0.1:9222" npm run inspect:questionnaire
```

The inspection script generates three core files:

1. `question_graph.json`: Compact question graph where each question lists its options, and each option lists its direct child questions.
2. `questionnaire_tree.json`: Raw debug tree recording probe edges, state changes, and skipped probes.
3. `question_graph.html`: Interactive local HTML visualization for manual inspection.

5.1.1 Google Questionnaire Structural Features

The Google Play IARC questionnaire exhibits the following structural characteristics:

1. **Dynamic Tree-Structured Branch Dependencies**: The questionnaire is a nested hierarchical tree rather than a flat form. A user's answer to a parent question (e.g., "Does the app contain violent content?") dynamically triggers or hides subsequent sub-questions. This parent-child dependency leads to distinct sets of reachable questions depending on the answers selected.
2. **Diverse Interaction Controls and State Constraints**: The form primarily consists of Radio Buttons and Checkboxes. Once a Radio Button is selected, it cannot be directly deselected to return to an empty state; it can only be changed by selecting another option in the same group. Checkboxes can be toggled freely between true and false.

3. **Stable Question and Option Identifiers**: To build an accurate graph representation, the script generates a unique, stable hash Key for each question based on its question type, prompt text, and set of available options, rather than relying on volatile DOM hierarchies or dynamic IDs. This ensures that a question's ID remains identical across different branches.

5.1.2 DFS Structure Inspection Algorithm

To automatically reconstruct the questionnaire's dynamic topology, we implemented a depth-first search (DFS) inspection script in `scripts/inspect_questionnaire.mjs`. The core execution process is as follows:

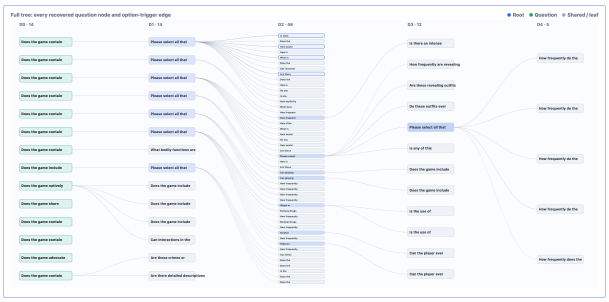
1. **Frontier-Based Visibility Detection**: Starting from the category selection screen, the script enters the questionnaire. It only probes questions that are currently visible on the page and lie within the current DFS search frontier, preventing invalid interactions.

2. **Option-Triggered Observation**: For each focused question, the script clicks its options one by one. After each click, it pauses for a rendering buffer (`PAUSE_MS`) and observes the newly appeared or disappeared sub-questions. If clicking an option displays new questions, they are added as children of that specific option in `question_graph.json`.

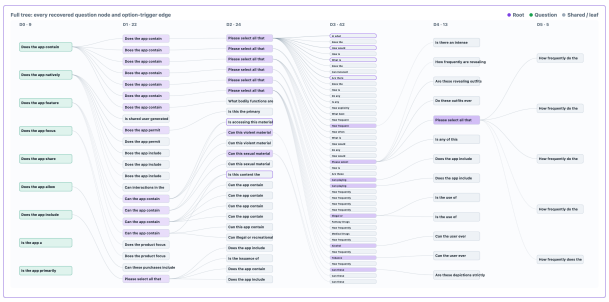
3. **State Recovery and Backtracking**: When a branch is fully explored and backtracking is required, the script restores the questionnaire state. Checkboxes are simply unchecked. For radio buttons, which cannot be deselected, the script either performs local restoration using recorded states or, if necessary, discards the draft, returns to the category selection page, and replays the path from the root to ensure state consistency.

4. **Graph Output and Visualization**: After traversal, the script clean and normalizes the data into `question_graph.json` and the detailed trace file `questionnaire_tree.json`.

Using this inspection tool, we exported the questionnaire topology for the supported categories. Figure 2 shows the structures of the Game and All Other App Types categories. The All Other App Types category consists of 115 questions with a wider and flatter branch structure, whereas the Game category has 91 questions with deeper, highly specialized sub-branches (e.g., gambling simulations, virtual currencies). Figure 3 shows a localized four-level question branch exemplifying sub-question triggers.



(a) Game Category Questionnaire Topology



(b) All Other App Types Topology

Figure 2: Inspected question structures for two typical categories.

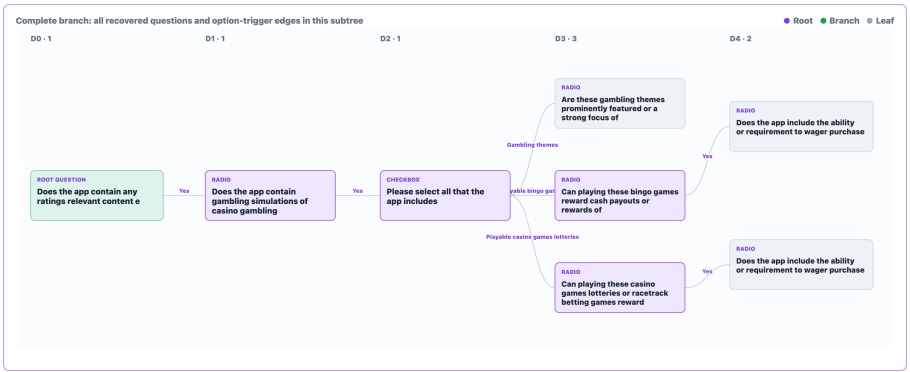


Figure 3: Example of localized sub-question dependencies (at depth 4).

The project covers three application categories:

- All Other App Types;
- Game;
- Social or Communication.

Each category triggers a distinct question subtree. For instance, the Game category is more likely to ask about virtual item purchases, simulated gambling, and in-game awards, whereas the Social or Communication category focuses on user-generated content, location sharing, and communication features. The All Other App Types category serves as the general baseline.

5.2 Sampling Strategy

Exhaustively combining all options results in an exponential sample space, with many combinations yielding redundant or low-information paths. To compile a diverse and informative training set, we design a multi-tiered sampling strategy:

1. **Baseline Samples**: Generates low-risk default paths for each category to establish a baseline.
2. **Structural Samples**: Systematically varies options along the questionnaire branches to cover all parent-child pathways.
3. **Risk-Driven Samples**: Generates dense samples around high-risk elements such as violence, gore, sexual content, drugs, and cash payouts.
4. **Borderline Samples**: Targets transitional categories (Everyone 10+, Teen, Mature 17+) to resolve decision boundaries.
5. **Coverage-Driven Samples**: Identifies infrequently selected options and prioritizes generating combinations to cover them.
6. **Gap-Filling Samples**: Dynamically computes missing or low-frequency reachable paths and injects target samples to fill gaps.

This strategy is highly suited to tree-structured questionnaires compared to random sampling: it avoids massive duplicates of low-risk samples and actively targets high-risk branches and boundary states.

6 Automated Data Collection

6.1 Automated Workflow

The collection script submits questionnaires automatically in the Google Play Developer Console. The core steps are:

1. Connects to a Chrome instance with an active login session;
2. Navigates to or restores the content rating questionnaire page;
3. Selects the target application category and accepts terms;
4. Answers all visible questions based on the generated sample;
5. Handles asynchronous rendering, loading spinners, and modal overlays;
6. Saves the draft and proceeds to the rating Summary page;
7. Parses the generated age ratings, content descriptors, and interactive elements;
8. Records the final results, execution evidence, errors, and run parameters;
9. References checkpoints to continue the batch or retry failed submissions.

6.2 Core Automation Code

The code block below demonstrates how the script connects to Chrome via CDP and reuses the open tab. This prevents Google login blocks and slot mismatches.

Listing 2: CDP connection and page reuse logic

```
async function openBrowserPage() {
  const browser = await chromium.connectOverCDP(config.cdpUrl);
  const context = browser.contexts()[0] || await browser.newContext();
  const existing = context.pages().find((candidate) =>
    candidate.url().includes('/content-rating')
  );
  const page = existing || context.pages()[0] || await context.newPage();
  return {
    page,
    close: async () => {
      if (typeof browser.disconnect === 'function') browser.disconnect();
    },
  };
}
```

The automation script also features robust state recovery. If a page gets stuck in a loading state, throws an error, or displays an overlay, the script detects the current UI state and recovers (e.g., by reloading, backtracking, or restarting the draft) to maintain autonomous collection.

6.3 Data Quality and Collection Failures

The final cleaned dataset contains 1,265 valid samples. During collection, 770 failures, 30 validation quarantine events, and 13 rating conflicts were recorded. These events are saved in `failures.jsonl`, `validation_quarantine.jsonl`, and `rating_conflicts.jsonl` to support debugging and sampling adjustments.

Table 5: Data Quality and Collection Statistics

Metric	Count
Cleaned Valid Samples	1,265
Excluded Samples in Task 3	0
Training Samples	1,012
Testing Samples	253
Standardized Question Answer Columns	164
Category-Question Unique Combinations	218
Territorial Rating Label Columns	10
Upstream Quarantined Samples	30
Upstream Rating Conflict Samples	13
Upstream Execution Failure Events	770

7 Data Cleaning and Dataset Analysis

7.1 Cleaning Principles

The goal of data cleaning is to construct a clean tabular format for modeling while strictly preventing label leakage. The cleaning pipeline consists of:

- Verifying schema versions and essential columns;
- Validating radio button value compliance;
- Corroborating explicit answers against the full questionnaire state;
- Verifying path reachability using the question graph to filter out impossible states;
- Confirming that territorial ratings match the category's expected output;
- Deduplicating sample IDs and identical answer combinations;
- Filtering out questionnaire states that produced conflicting ratings across runs;
- Encoding structural missing states into unique non-null sentinels.

No statistical imputation is performed. "Missing" values in this task represent structural properties of the questionnaire rather than random data loss. For instance, `__INACTIVE__` denotes a question that was not triggered by parent answers; `__NOT_APPLICABLE__` denotes a question/rating that is irrelevant to the category; and `__NONE__` indicates that a visible checkbox question was left unchecked. These states contain key predictive signals.

7.2 Train/Test Split

The dataset is partitioned using a deterministic 80/20 holdout split stratified by application category and North American ESRB rating. The seed is fixed to `osn-lab2-task3-v1`. This setup:

1. Ensures that the rating label distribution remains identical across the train and test sets;
2. Guarantees exact reproducibility across different execution environments.

Table 6: Dataset Category Distribution

Application Category	Total	Train Set	Test Set	Test Ratio
All Other App Types	616	492	124	20.13%
Game	633	507	126	19.91%
Social or Communication	16	13	3	18.75%
Total	1,265	1,012	253	20.00%

Table 7: North American ESRB Rating Distribution

Age Rating	Total	Train Set	Test Set	Test Ratio
Everyone	182	146	36	19.78%
Everyone 10+	138	110	28	20.29%
Teen	339	271	68	20.06%
Mature 17+	365	292	73	20.00%
Adults only 18+	241	193	48	19.92%
Total	1,265	1,012	253	20.00%

7.3 Data Distribution Visualization

Figure 4 shows the sample sizes across the age ratings. Everyone 10+ is the minority class, whereas Mature 17+ and Teen are the majority classes. This class imbalance affects the model's recall and F1-scores on intermediate ratings.

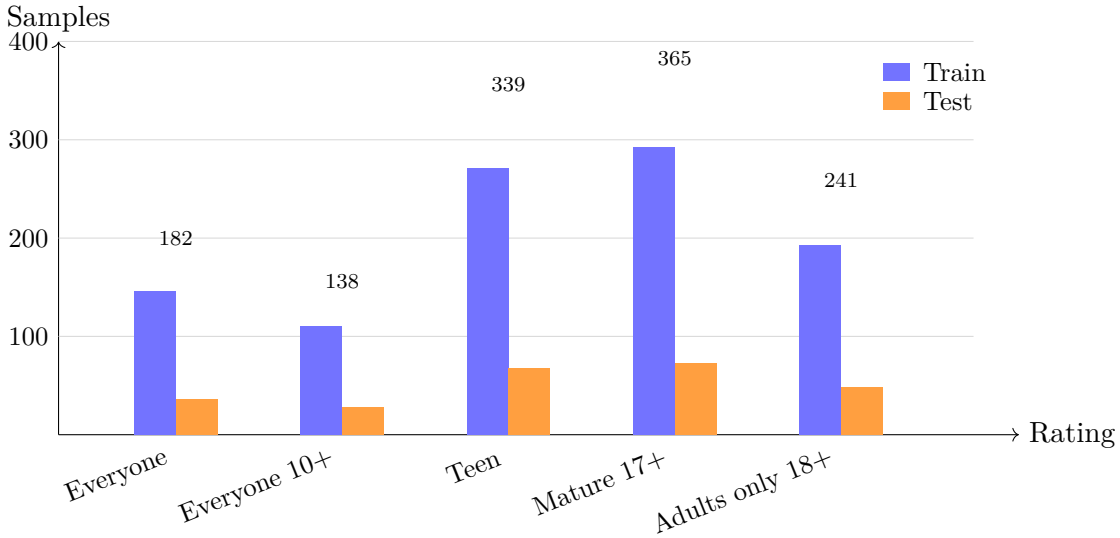


Figure 4: Sample distribution across ESRB rating classes. The numbers represent the total sample count for each rating.

8 Feature Engineering

8.1 Input and Output Definition

The prediction target is `rating__north_america`. The model input consists exclusively of:

- `category`: The application category;
- `answer__*`: The answer values for questionnaire items.

To prevent label leakage, all other territorial rating columns (e.g., European PEGI, German USK, etc.) are strictly excluded. Even when only predicting the North American rating, using other ratings would grant the model unfair information, since those ratings are generated simultaneously by the same black-box system.

8.2 Encoding Scheme

The training dataset from Task 3 is split further to create a validation set. The final split for feature engineering is:

- Train Set: 810 samples;
- Validation Set: 202 samples;
- Test Set: 253 samples (identical to the Task 3 test set).

The encoding scheme is structured as follows:

1. **Category and Radio fields**: Encoded using one-hot encoding.
2. **Checkbox fields**: Checkbox lists are split by the separator `||` and encoded using multi-hot encoding.
3. **Structural sentinels**: The strings `__INACTIVE__`, `__NOT_APPLICABLE__`, and `__NONE__` are treated as valid categorical categories.
4. **Unseen values**: Any new answer values appearing in the validation or test sets that were not observed in the training set are ignored and recorded in `unseen_values.json`.

Table 8: Feature Engineering Scale

Metric	Count
Raw Feature Columns	165
Category / Radio One-Hot Fields	133
Checkbox Multi-Hot Fields	32
Encoded Feature Columns (Dimensions)	758
Train Samples (Task 4)	810
Validation Samples (Task 4)	202
Test Samples (Final)	253

8.3 Core Feature Engineering Code

The Python code snippet below demonstrates our sentinel definition and deterministic validation split logic.

Listing 3: Labels and structural state definitions

```
MISSING_SENTINELS = {"__INACTIVE__", "__NOT_APPLICABLE__", "__NONE__"}
CHECKBOX_SEPARATOR = " || "
DEFAULT_LABEL_ORDER = [
    "Everyone",
    "Everyone 10+",
    "Teen",
    "Mature 17+",
    "Adults only 18+",
]
```

Listing 4: Deterministic validation splitting stratified by category and rating

```
def deterministic_validation_split(train_df, target_column, validation_ratio, seed):
    strata = train_df["category"].astype(str) + " | " + train_df[target_column].astype(str
        )
    train_indices = []
    val_indices = []

    for _, group in train_df.assign(_stratum=strata).groupby("_stratum", sort=True):
        records = []
        for idx, row in group.iterrows():
            key = f"{seed}|{row['sample_id']}|{row['_stratum']}"
            records.append((stable_hash(key), idx))
        records.sort()

        n = len(records)
        if n <= 1:
            n_val = 0
        else:
            n_val = max(1, round(n * validation_ratio))
            n_val = min(n_val, n - 1)

        val_indices.extend(idx for _, idx in records[:n_val])
        train_indices.extend(idx for _, idx in records[n_val:])
```

9 Model Training and Parameter Tuning

9.1 Model Selection

We trained and compared four models spanning linear classification, Bagging ensemble trees, and Boosting ensemble trees:

Table 9: Selected Models and Underlying Hypotheses

Model	Type	Selection Rationale
Logistic Regression	Linear Multi-Class	One-hot and multi-hot features produce high-dimensional sparse representations where linear models provide strong baselines. L1 regularization can select crucial features.
Random Forest	Bagging Trees	Learns non-linear interactions and is invariant to feature scaling; serves as a strong tree baseline.
LightGBM	Boosting Trees	Fast training and suitable for tabular data; offers rich control parameters (depth, leaves, child samples) to prevent overfit.
XGBoost	Boosting Trees	State-of-the-art tabular model; supports column sampling, early stopping, and L1/L2 regularization on leaves.

9.2 Logistic Regression and Random Forest

The training script for these two models is located at `task5_2_logistic_regression\&random_forest/train_models.py`. We applied balanced class weights to address the rating imbalance. The final hyperparameters are:

Listing 5: Logistic Regression and Random Forest definitions

```
models = {
    "logistic_regression": LogisticRegression(
        C=10.0,
        l1_ratio=1.0,
        tol=1e-4,
        solver="saga",
        max_iter=5000,
        class_weight=class_weight_by_id,
        random_state=42,
    ),
    "random_forest": RandomForestClassifier(
        n_estimators=500,
        max_depth=None,
        min_samples_leaf=1,
        max_features=0.3,
        min_samples_split=2,
        class_weight=class_weight_by_id,
        random_state=42,
        n_jobs=-1,
    ),
}
```

Logistic Regression uses L1 regularization (SAGA solver), enforcing sparsity across the multi-class coefficient weights. Since the feature space is sparse, L1 regularization nullifies irrelevant question choices, isolating the select few that determine rating transitions.

Random Forest's default parameter `max_features='sqrt'` performs poorly here, as it only samples 27 features per split, which frequently misses key question triggers in a 758-dimensional space. By tuning `max_features` to 0.3 (allowing each split to evaluate 228 features), test set Accuracy improved from 0.617 to 0.783, and Macro F1 rose from 0.624 to 0.797.

9.3 LightGBM and XGBoost

LightGBM and XGBoost training and tuning are documented in `task5_1_lightgbm\&xgboost/ana1.ipynb`. We applied regularized search to prevent overfitting:

Table 10: LightGBM and XGBoost Hyperparameters

Model	Parameter	Optimized Value / Description
LightGBM	`max_depth`	6 (constrains tree depth)
LightGBM	`num_leaves`	15 (restricts leaf size)
LightGBM	`min_child_samples`	15 (requires minimum samples per leaf)
LightGBM	`learning_rate`	0.1
XGBoost	`max_depth`	6 (limits individual tree complexity)
XGBoost	`subsample`	1.0
XGBoost	`colsample_bytree`	0.8 (samples 80% of features per tree)
XGBoost	`learning_rate`	0.1

10 Experimental Results and Model Comparison

10.1 Overall Performance

Table 11 compares the four models on the test set. Logistic Regression significantly outperforms the tree-based models. XGBoost ranks second, Random Forest third, and LightGBM fourth.

Table 11: Model Performance Comparison on the Test Set

Model	Test Accuracy	Test Macro F1	Test Macro AUC
Logistic Regression	**0.921**	**0.918**	**0.977**
XGBoost	0.814	0.810	0.962
Random Forest	0.783	0.797	0.945
LightGBM	0.680	0.680	0.924

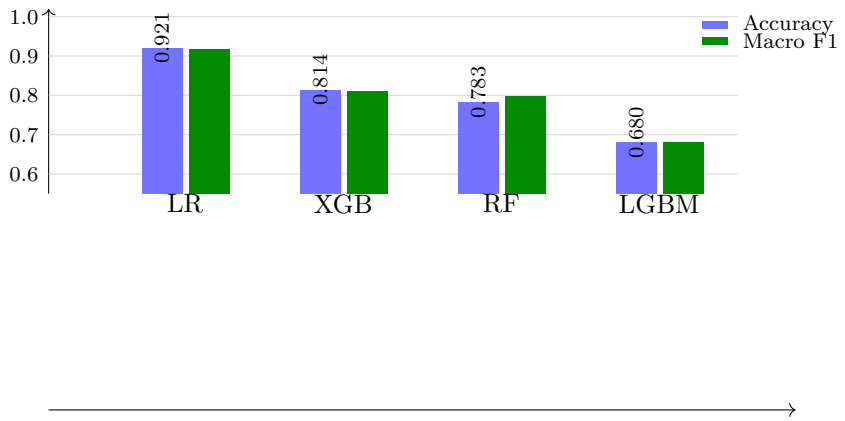


Figure 5: Comparison of Accuracy and Macro F1 across the four models.

10.2 Generalization Gap

Table 12 compares validation and test set performance for Logistic Regression and Random Forest. Both models display minor generalization gaps, indicating no severe overfitting.

Table 12: Validation and Test Performance Comparison

Model	Val Accuracy	Val Macro F1	Test Accuracy	Test Macro F1
Logistic Regression	0.936	0.928	0.921	0.918
Random Forest	0.812	0.811	0.783	0.797

10.3 Classification Report of the Best Model

Table 13 displays the class-level performance of our best model, Logistic Regression. Predictions are robust for Everyone, Mature 17+, and Adults only 18+. The Everyone 10+ category has the lowest F1-score due to its small sample size and transitional semantics between Everyone and Teen.

Table 13: Logistic Regression Test Set Classification Report

Class	Precision	Recall	F1-score	Support
Everyone	0.921	0.972	0.946	36
Everyone 10+	0.889	0.857	0.873	28
Teen	0.855	0.956	0.903	68
Mature 17+	0.957	0.918	0.937	73
Adults only 18+	1.000	0.875	0.933	48
Macro Avg	0.924	0.916	0.918	253
Weighted Avg	0.925	0.921	0.921	253

10.4 Confusion Matrix Analysis

The confusion matrix for Logistic Regression on the test set is shown in Figure 6. Errors are confined to adjacent or semantically overlapping categories:

- 4 Everyone 10+ samples are predicted as Teen;
- A few Teen samples are misclassified as Everyone, Everyone 10+, or Mature 17+;
- 4 Mature 17+ samples are predicted as Teen;
- 3 Adults only 18+ samples are predicted as Teen, and 2 as Mature 17+.

These errors are localized along the age spectrum rather than scattered randomly, indicating the model learns a coherent grading scale.

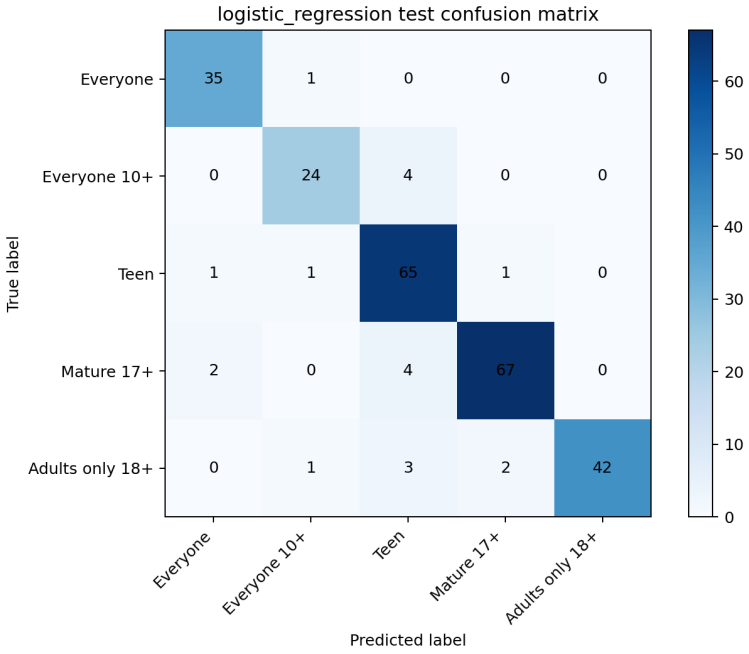


Figure 6: Logistic Regression test set confusion matrix.

Random Forest's confusion matrix is shown in Figure 7. Compared to Logistic Regression, Random Forest exhibits higher confusion between Teen and Mature 17+, reflecting that tree models require more samples to learn fine-grained boundaries on sparse datasets.

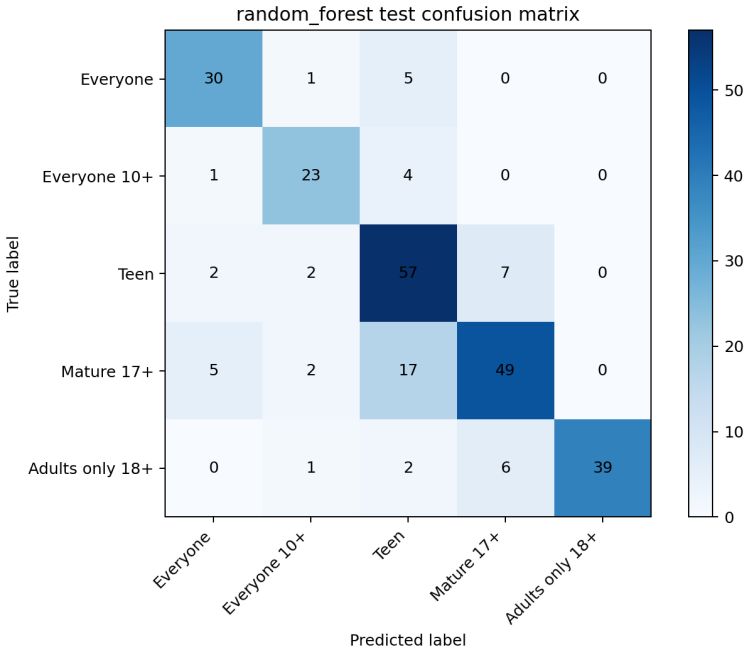
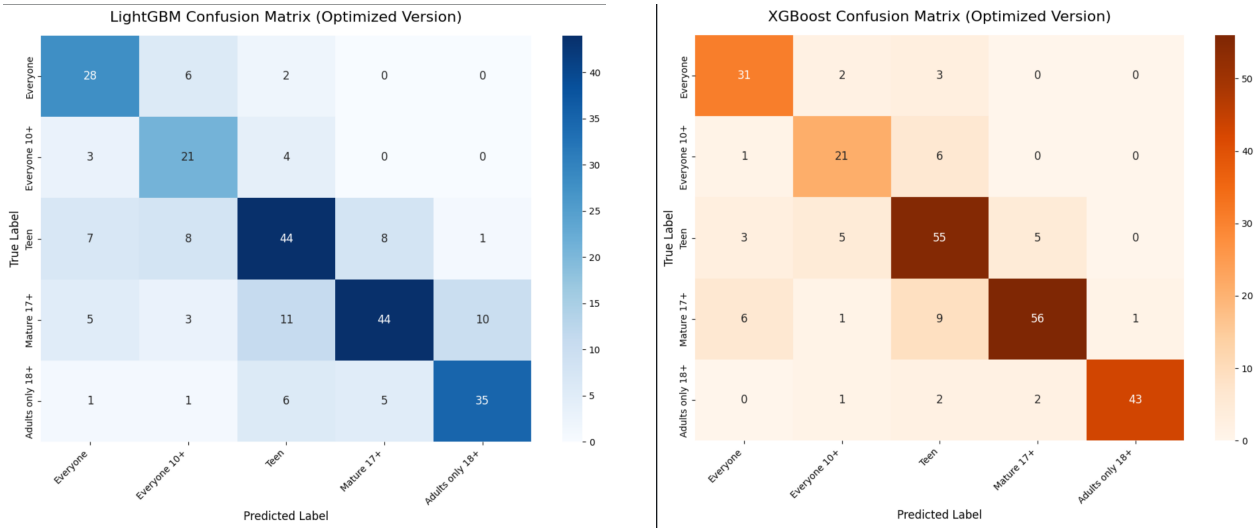


Figure 7: Random Forest test set confusion matrix.

The confusion matrices for LightGBM and XGBoost are shown in Figure 8. XGBoost outperforms LightGBM but remains inferior to L1-regularized Logistic Regression.



(a) LightGBM (b) XGBoost

Figure 8: LightGBM and XGBoost confusion matrices.

11 In-depth Analysis

11.1 Why L1 Logistic Regression Performs Best

The key experimental finding is that L1-regularized Logistic Regression outperforms Random Forest, LightGBM, and XGBoost. This aligns with the properties of the encoded feature representation.

The questionnaire answers are encoded into a 758-dimensional sparse matrix. Each sample activates only a small fraction of features. Google Play's age rating rules likely consist of strong, additive threshold triggers (e.g., presence of gore, frequent sexual themes, drug use, cash gambling). In a one-hot feature space, such triggers translate to linear or piecewise linear boundaries.

The advantages of L1 regularization in this context include:

1. **Sparsity Alignment**: Zeroes out irrelevant question choices.
2. **Mitigating Overfitting**: Given 1,265 samples and 758 features, sparse coefficients reduce the model's capacity, preventing overfitting.
3. **High Interpretability**: Non-zero coefficients correspond to concrete questionnaire items, revealing the underlying decision logic.
4. **Direct Factor Isolation**: If a single option determines an age jump, a linear classifier directly assigns it a high weight.

Conversely, tree-based models must perform recursive coordinate-aligned splits to replicate additive logic, which demands larger training volumes. LightGBM's leaf-wise growth on small, highly sparse tables easily results underfit or unstable splits.

11.2 Core Influential Factors

SHAP values and L1 coefficients highlight the most influential question signals, summarized in Table 14.

Table 14: Primary Questionnaire Signals Influencing Age Ratings

Rank	Signal	Explanation
1	Blood/Gore Severity	Higher severity drives ratings towards Teen, Mature 17+, or Adults only 18+.
2	Sexual Themes and Nudity	Major triggers for higher age classes.
3	Erotic/Suggestive Themes Branch	Entry into these sub-questions indicates risk; inactive states conversely signal low risk.
4	Illicit or Recreational Drugs	Increases ratings and defines the Teen vs. Mature 17+ boundary.
5	Gambling, Simulated Gambling	Simulated gambling or cash rewards push ratings significantly higher.
6	User-Generated Interaction	Relevant for Social or Communication categories, though limited by sample size.

These factors correspond directly to established media content rating guidelines: content containing violence, sexuality, drugs, or gambling is restricted to older audiences.

11.3 Significance of Structural Missingness

While classical tabular pipelines treat missing values as noise to be imputed, `__INACTIVE__`, `__NOT_APPLICABLE__`, and `__NONE__` represent structural information in this task. Specifically, `__INACTIVE__` means a question was bypassed because its parent question received a negative answer (e.g., bypasses drug subquestions if the app contains no drugs).

Imputing these states would erase this implicit logic. By encoding them as categories, the models directly utilize the bypassed paths. This choice is validated by feature importance rankings, where `__INACTIVE__` states regularly appear as strong negative risk indicators.

11.4 Questionnaire Redundancy and Minimal Feature Sets

Feature sub-sampling experiments reveal that a small subset of high-weight features is sufficient to replicate the baseline model's performance:

Table 15: Feature Count vs. Test Accuracy

Selected Features	Test Accuracy	Relative Performance
10	0.435	47%
50	0.625	68%
200	0.933	101%
758	0.921	100%

This demonstrates substantial redundancy in the Google Play questionnaire. Many questions do not affect the North American ESRB rating, or only resolve rare boundary cases. This allows us to:

1. Build compact estimators to predict ratings with minimal question filling;
2. Understand the high-impact rules to help developers identify critical risk areas.

11.5 Gray Areas in Rating Boundaries

Model uncertainty is concentrated on the boundaries of intermediate ratings, such as Teen vs. Mature 17+ and Everyone 10+ vs. Teen. This stems from the subjective nature of these ratings, which depend on fine-grained differences in frequency and presentation. Everyone and Adults only 18+ represent extreme ends of the risk spectrum and are clearly separated. The errors in the confusion matrix are confined to adjacent rating classes, indicating the model learns a smooth risk scale.

12 Experimental Challenges and Solutions

12.1 Data Collection Challenges

Table 16: Data Collection Challenges and Solutions

Challenge	Impact	Solution
Google Login Protection	Playwright default headless browser fails login challenges, blocking access to Play Console.	Connected via CDP to a real Chrome profile with an active login session.
Account Slot Mismatch	Google URLs map accounts dynamically (e.g., `/u/0`, `/u/2`). Incorrect accounts lack project permissions.	Reuses open tabs containing '/content-rating' and supports custom URLs via `PLAY_CONSOLE_URL`.
Unstable Save State	The page can hang on loading spinner after draft saving, missing the rating Summary page.	Implemented state loops, timeout handlers, and recovery rules.
Modal Overlays	Pop-up notifications block automated click paths.	Wrote `dismissOverlays` to close dialogs and clear the viewport.
Dependency Violation	Sampling targets may request hidden questions, causing collection script failure.	Verified reachable path logic via the question graph before execution.
Duplicate	Semantic duplication or conflicting rating outputs.	Deduplicated samples at the data stage and exported conflicts to `rating_conflicts.jsonl`.

12.2 Modeling Challenges

Table 17: Modeling Challenges and Solutions

Challenge	Impact	Solution
Class Imbalance	Everyone 10+ has few samples, biasing models towards Teen/Mature 17+.	Applied `class_weight='balanced'` or equivalent class multipliers.
Target Leakage	Bounding ratings from other regions creates artificial accuracy.	Excluded all rating columns, keeping only `category` and `answer__*`.
High-Dimensional Sparsity	Default tree settings fail to sample informative splits.	Scaled `max_features` to 0.3 for Random Forest and restricted tree depth with column sub-sampling for boosting models.
LightGBM Underfitting	Yields an F1-score of only 0.680 on the test set.	Evaluated parameter configurations, referencing the mismatch between tree partitioning and sparse linear logic.
Borderline Ambiguity	Confusion on Everyone 10+ / Teen / Mature 17+.	Monitored Macro F1 and analyzed error localization using confusion matrices.

13 Group Division of Labor

This project spans web automation, data engineering, feature modeling, and analysis. The group adopted a modular workflow. The role distribution is detailed in Table 18.

Table 18: Group Member Division of Labor

Member	Core Responsibilities
Yuhan Chen	Project repository maintenance; Google Play Developer Console automation pipeline; questionnaire structure inspection, sample collection, and database maintenance.
Jiashun Shi	Raw data processing and pipeline engineering: deduplication, missing value checks, quarantined outlier removal, column name standardization, and train/test splits.
Bofan Chen	Feature engineering: raw question mapping, structural sentinel management, one-hot / multi-hot encoding, and feature/label dictionaries.
Zihan Chen	LightGBM and XGBoost training and parameter tuning, boosting model evaluation, and comparative performance analysis.
Zixuan Wang	Logistic Regression and Random Forest model training, parameter sweeps, model serialization, and metric reporting.

The project steps are highly connected: collection results drive feature updates, and model errors suggest areas requiring further targeted sampling. The group operated as a collaborative closed loop.

14 Major Project Findings

Our experimental analysis yields the following conclusions:

1. **Google Play Age Ratings are Highly Predictable**: Logistic Regression achieved a test Accuracy of 0.921 and a Macro F1 of 0.918, indicating a stable mapping between questionnaire answers and ESRB ratings.
2. **Linear Models Fit High-Dimensional Sparse Inputs Best**: L1-regularized Logistic Regression outperformed tree architectures, demonstrating that content rules act as sparse linear triggers in one-hot space.
3. **Violence, Sex, Drugs, and Gambling are Key Triggers**: These elements have the highest predictive weight, aligning with established rating standards.
4. **Questionnaire Topology Contains Information**: Structural missingness states like `__INACTIVE__` are predictive features that represent untriggered sub-branches, and should not be imputed.
5. **Substantial Redundancy Exists**: The model matches baseline performance using only 200 of the 758 features.
6. **Transitional Boundaries present High Ambiguity**: Misclassifications are restricted to adjacent categories (Everyone 10+ vs. Teen vs. Mature 17+).

15 Limitations and Future Directions

15.1 Current Limitations

1. **Sparse Social/Communication Samples**: Only 16 samples were collected for this category (3 in the test set), preventing reliable modeling.
2. **Data Scale**: While satisfying the 1,000-sample limit, 1,265 samples are relatively small for a 758-dimensional space.
3. **Temporal Drift**: The rating logic is a black-box system that Google may update, and current rules reflect a snapshot of the console.
4. **North American Focus**: We focused on `rating__north_america`, leaving regional differences unanalyzed.
5. **Correlation vs. Causation**: L1 coefficients show statistical correlation rather than Google's internal causal code.

15.2 Future Directions

We propose the following directions:

- Expand sampling for the Social or Communication category;
- Use active learning to query samples near rating boundaries;
- Model other jurisdictions (PEGI, USK, ACB) and analyze cross-regional rules;
- Extract symbolic decision trees from high-weight L1 features;
- Calibrate model probabilities to output confidence levels for developers;
- Package scripts into a stable execution pipeline independent of interactive browser state.

16 Conclusion

This project reverse models the Google Play age rating questionnaire. Starting from dynamic questionnaire inspection, we designed a multi-tiered sampling strategy, automated submission and collection using Playwright, and compiled a dataset of 1,265 samples. We trained and compared Logistic Regression, Random Forest, LightGBM, and XGBoost on 758-dimensional features.

L1-regularized Logistic Regression achieved the highest accuracy, with 0.921 Accuracy, 0.918 Macro F1, and 0.977 Macro AUC on the test set. This confirms that the underlying rating system can be modeled with high accuracy using data-driven methods. Violence, drugs, sexuality, and gambling are the main drivers of ratings, and structural missingness represents key information.

Overall, the project successfully completed the data collection, feature engineering, modeling, comparison, and analysis requirements, providing insight into Google Play's age rating mechanism.

A Experiment Replication Commands

Listing 6: Main replication commands

```
# Install Node dependencies
npm install

# Inspect questionnaire structure
CDP_URL="http://127.0.0.1:9222" npm run inspect:questionnaire

# Run sampling
CDP_URL="http://127.0.0.1:9222" npm run run:samples

# Build and validate Dataset V2
npm run build:dataset-v2
npm run validate:dataset-v2

# Feature engineering
python task4_feature_engineering/scripts/build_features.py

# Train Logistic Regression and Random Forest
python "task5_2_logistic_regression&random_forest/train_models.py"
```

B Key Output Files

Table 19: Key Project Output Files

File Path	Description
`dataset_v2/samples.jsonl`	Primary dataset containing deduplicated samples.
`dataset_v2/coverage_report.json`	Distribution metrics for categories, ratings, and questions.
`task3_data_cleaning/output/clean_dataset.csv`	Full cleaned dataset.
`task3_data_cleaning/output/train.csv`	Task 3 training split.
`task3_data_cleaning/output/test.csv`	Task 3 test split.
`task4_feature_engineering/output/X_train.npz`	Sparse feature matrix for training.
`task4_feature_engineering/`	

File Path	Description
`output/feature_dictionary.csv`	Feature mapping back to raw questionnaire options.
`task4_feature_engineering/output/label_mapping.json`	Target rating indices.
`task5_2_logistic_regression\&random_forest/output/model_comparison.csv`	Metrics for Logistic Regression and Random Forest.
`task5_2_logistic_regression\&random_forest/output/metrics.json`	Run parameters and validation/test metrics.
`task5_2_logistic_regression\&random_forest/output/*.png`	Plot files for confusion matrices.
`task5_1_lightgbm\&xgboost/img/*.png`	Plot files for LightGBM and XGBoost confusion matrices.

C References

1. Google Play Developer Console: https://play.google.com/console/developers
2. Project Repository: https://github.com/CYH0415/OSN-Lab-Main
3. Playwright Documentation: https://playwright.dev/
4. scikit-learn Documentation: https://scikit-learn.org/
5. XGBoost Documentation: https://xgboost.readthedocs.io/
6. LightGBM Documentation: https://lightgbm.readthedocs.io/